

C PROGRAMMING

Unit-III: Control Statements and Arrays

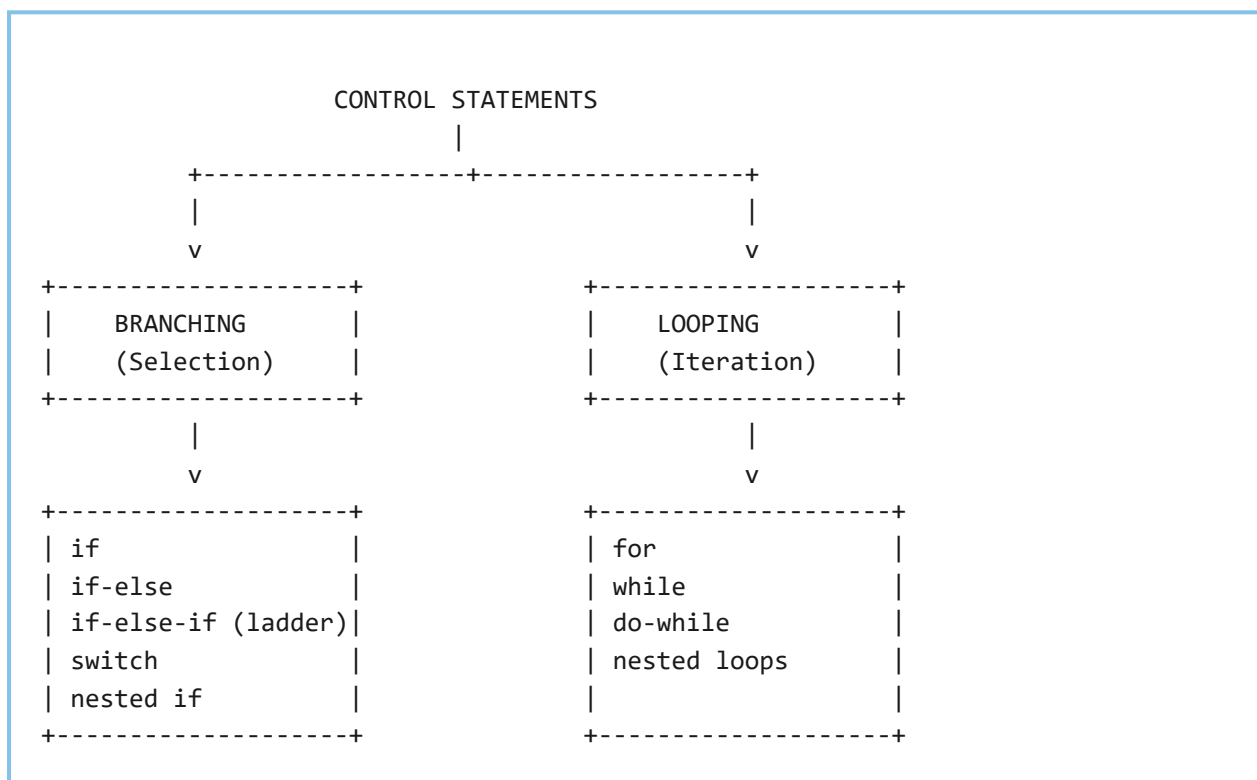
Comprehensive Study Notes

Topic 1: Control Statements

1.1 What are Control Statements?

Control statements are programming constructs that **control the flow of execution** of a program. They allow the program to make decisions, repeat tasks, and jump between different parts of the code.

1.2 Types of Control Statements



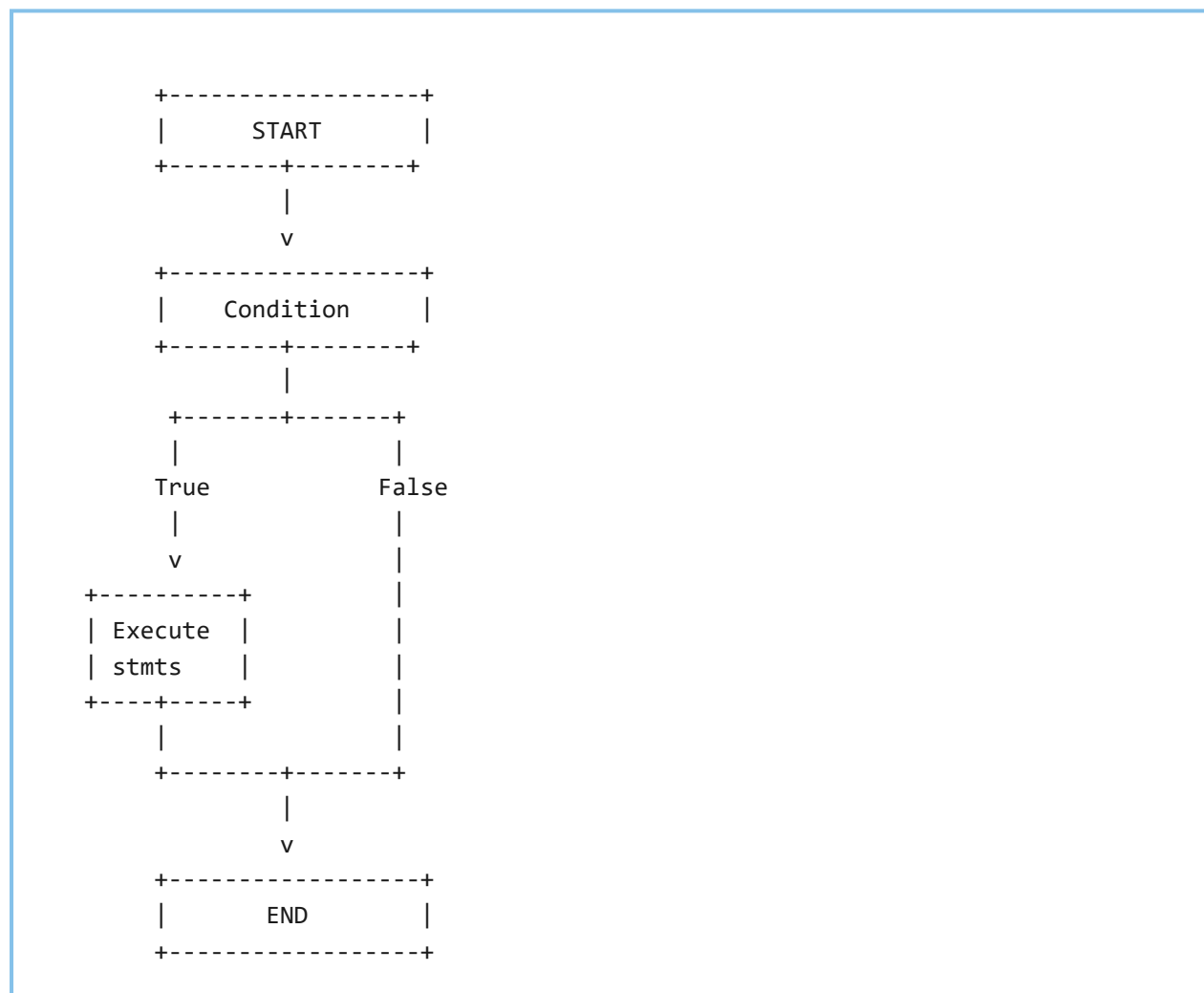
Topic 2: Branching Statements

2.1 if Statement

Syntax:

```
if (condition)
{
    // Statements to execute if condition is true
}
```

Flowchart:



Code:

```
#include <stdio.h>

int main()
{
    int age;

    printf("Enter your age: ");
    scanf("%d", &age);

    if (age >= 18)
    {
        printf("You are eligible to vote.\n");
    }

    printf("Thank you.\n");

    return 0;
}
```

Output:

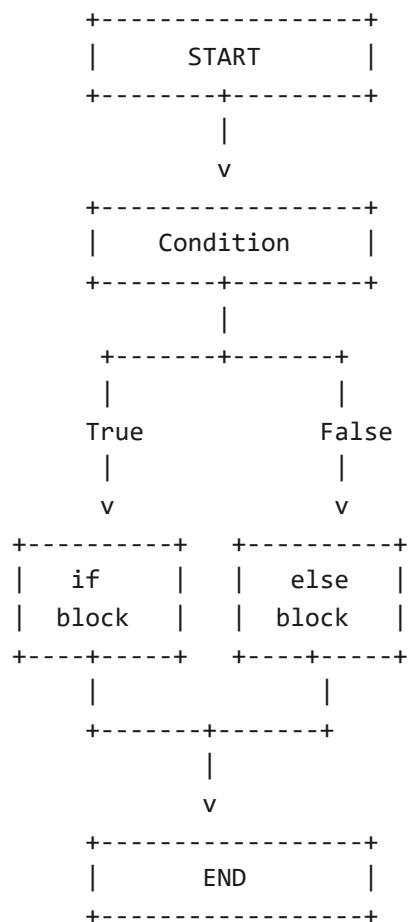
```
Enter your age: 20
You are eligible to vote.
Thank you.
```

2.2 if-else Statement

Syntax:

```
if (condition)
{
    // Statements if condition is true
}
else
{
    // Statements if condition is false
}
```

Flowchart:



Code:

```
#include <stdio.h>

int main()
{
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num % 2 == 0)
    {
        printf("%d is even.\n", num);
    }
    else
    {
        printf("%d is odd.\n", num);
    }

    return 0;
}
```

Output:

```
Enter a number: 7
7 is odd.
```

2.3 if-else-if Ladder

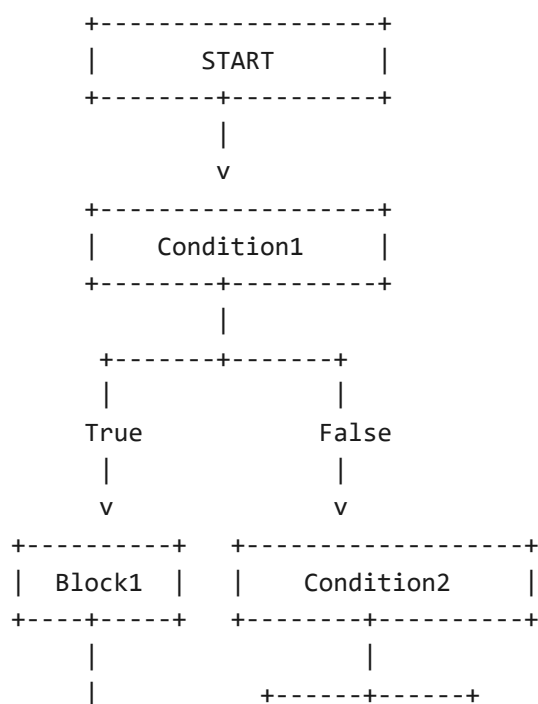
Syntax:

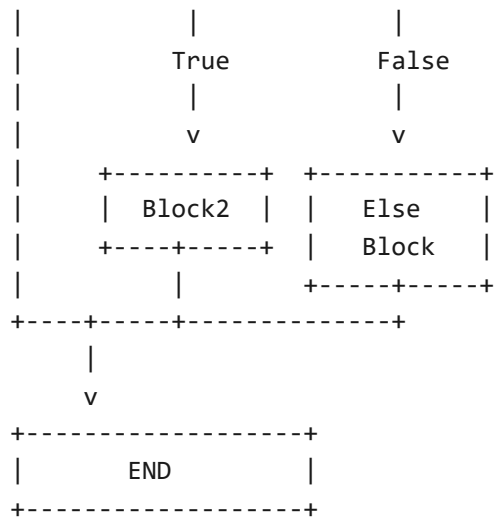
```

if (condition1)
{
    // Statements if condition1 is true
}
else if (condition2)
{
    // Statements if condition2 is true
}
else if (condition3)
{
    // Statements if condition3 is true
}
else
{
    // Statements if all conditions are false
}

```

Flowchart:





Code:

```

#include <stdio.h>

int main()
{
    int marks;

    printf("Enter your marks: ");
    scanf("%d", &marks);

    if (marks >= 90)
    {
        printf("Grade: A (Excellent)\n");
    }
    else if (marks >= 80)
    {
        printf("Grade: B (Very Good)\n");
    }
    else if (marks >= 70)
    {
        printf("Grade: C (Good)\n");
    }
    else if (marks >= 60)
    {
        printf("Grade: D (Average)\n");
    }
    else if (marks >= 50)
    {
        printf("Grade: E (Pass)\n");
    }
}

```



```
    }  
    else  
    {  
        printf("Grade: F (Fail)\n");  
    }  
  
    return 0;  
}
```

Output:

```
Enter your marks: 85  
Grade: B (Very Good)
```

2.4 Nested if-else

Syntax:

```
if (condition1)
{
    if (condition2)
    {
        // Statements when both true
    }
    else
    {
        // Statements when condition1 true, condition2 false
    }
}
else
{
    // Statements when condition1 false
}
```

Code:

```
#include <stdio.h>

int main()
{
    int a, b, c;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a > b)
    {
        if (a > c)
        {
            printf("%d is the largest.\n", a);
        }
        else
        {
            printf("%d is the largest.\n", c);
        }
    }
}
```

```
    }  
    else  
    {  
        if (b > c)  
        {  
            printf("%d is the largest.\n", b);  
        }  
        else  
        {  
            printf("%d is the largest.\n", c);  
        }  
    }  
  
    return 0;  
}
```

Output:

Enter three numbers: 10 25 15
25 is the largest.

Topic 3: Switch Statement

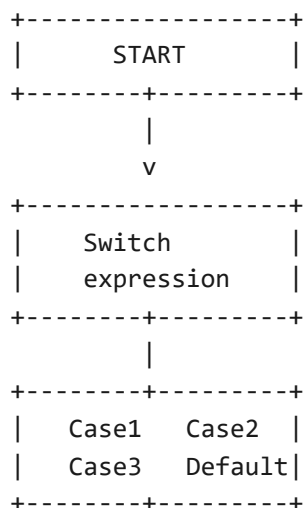
3.1 What is Switch Statement?

Switch statement is a **multi-way branching** statement that tests a variable against a list of values.

3.2 Switch Syntax

```
switch (expression)
{
    case value1:
        // Statements
        break;
    case value2:
        // Statements
        break;
    ...
    default:
        // Statements if no match
}
```

Flowchart:



```
      |  
      v  
+-----+  
|   Execute   |  
| matching case |  
+-----+  
      |  
      v  
+-----+  
|       END       |  
+-----+
```

Code:

```
#include <stdio.h>  
  
int main()  
{  
    int day;  
  
    printf("Enter day number (1-7): ");  
    scanf("%d", &day);  
  
    switch (day)  
    {  
        case 1:  
            printf("Monday\n");  
            break;  
        case 2:  
            printf("Tuesday\n");  
            break;  
        case 3:  
            printf("Wednesday\n");  
            break;  
        case 4:  
            printf("Thursday\n");  
            break;  
        case 5:  
            printf("Friday\n");  
            break;  
        case 6:  
            printf("Saturday\n");  
            break;  
        case 7:  
            printf("Sunday\n");  
    }
```

```
        break;
    default:
        printf("Invalid day number!\n");
    }

    return 0;
}
```

Output:

```
Enter day number (1-7): 3
Wednesday
```

3.3 Rules for Switch Statement

Rule	Description
1	Expression must be integer or character
2	Case values must be constants
3	Case values must be unique
4	break is optional (fall-through if omitted)
5	default is optional
6	Multiple cases can share same statements

3.4 Program: Switch with Character

```
#include <stdio.h>

int main()
{
    char choice;
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Enter operation (+, -, *, /): ");
    scanf(" %c", &choice);

    switch (choice)
    {
        case '+':
            printf("%d + %d = %d\n", a, b, a + b);
            break;
        case '-':
            printf("%d - %d = %d\n", a, b, a - b);
            break;
        case '*':
```

```
        printf("%d * %d = %d\n", a, b, a * b);
        break;
    case '/':
        if (b != 0)
            printf("%d / %d = %.2f\n", a, b, (float)a / b);
        else
            printf("Division by zero!\n");
        break;
    default:
        printf("Invalid operation!\n");
    }

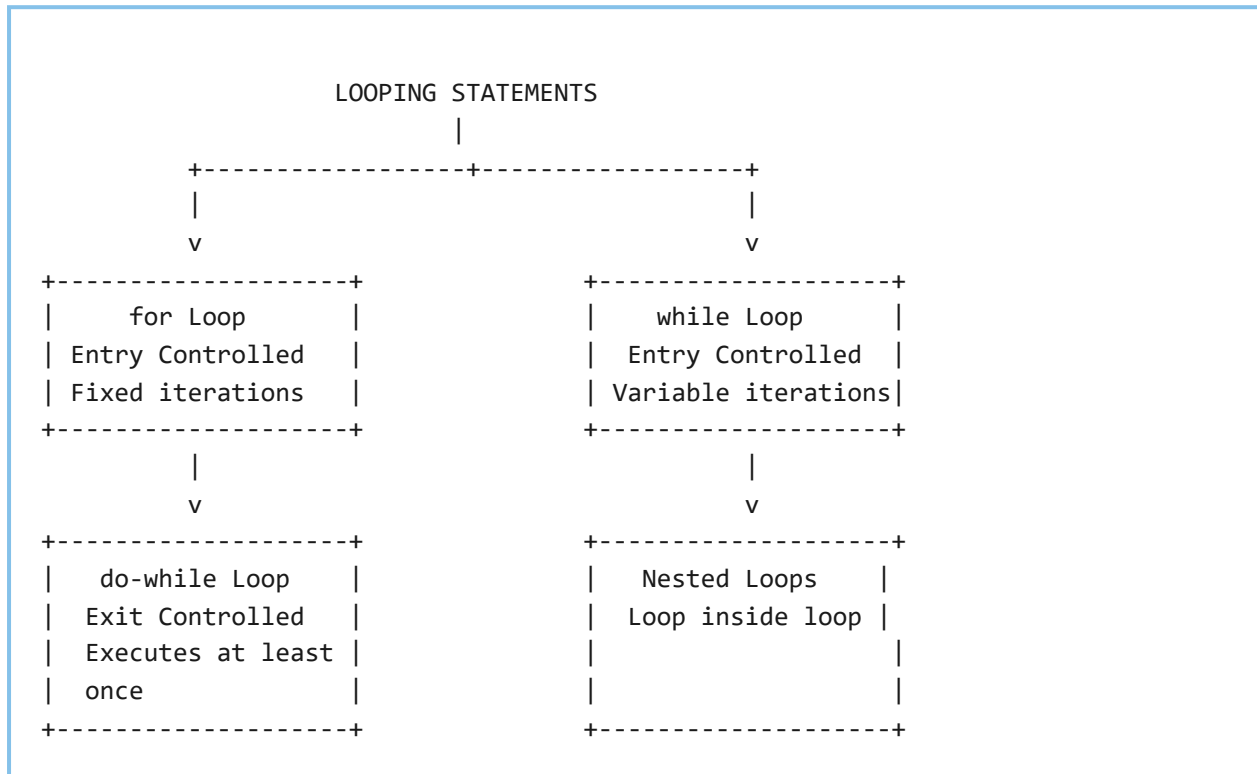
    return 0;
}
```

Output:

```
Enter two numbers: 10 5
Enter operation (+, -, *, /): *
10 * 5 = 50
```


Topic 4: Looping Statements

4.1 Types of Loops



4.2 for Loop

Syntax:

```

for (initialization; condition; increment/decrement)
{
    // Statements to repeat
}

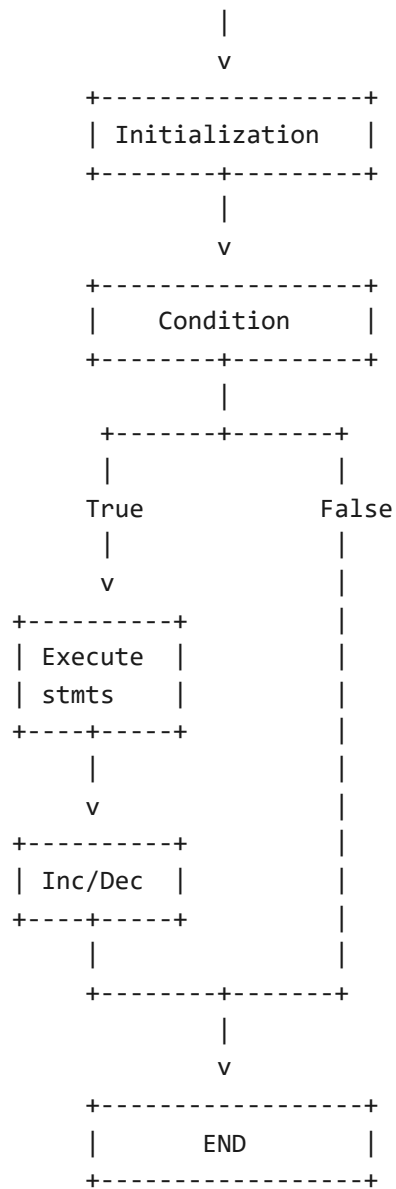
```

Flowchart:

```

graph TD
    START[START]

```



Code:

```
#include <stdio.h>

int main()
{
    int i;

    // Print numbers from 1 to 10
    printf("Numbers from 1 to 10:\n");
    for (i = 1; i <= 10; i++)
    {
        printf("%d ", i);
    }
    printf("\n");

    // Print even numbers from 2 to 20
    printf("Even numbers from 2 to 20:\n");
    for (i = 2; i <= 20; i += 2)
    {
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

Output:

```
Numbers from 1 to 10:
1 2 3 4 5 6 7 8 9 10
Even numbers from 2 to 20:
2 4 6 8 10 12 14 16 18 20
```

4.3 Program: Sum of n Numbers using for Loop

```
#include <stdio.h>

int main()
{
```

```
int n, i, sum = 0;

printf("Enter a positive integer: ");
scanf("%d", &n);

for (i = 1; i <= n; i++)
{
    sum += i; // sum = sum + i
}

printf("Sum of numbers from 1 to %d is: %d\n", n, sum);

return 0;
}
```

Output:

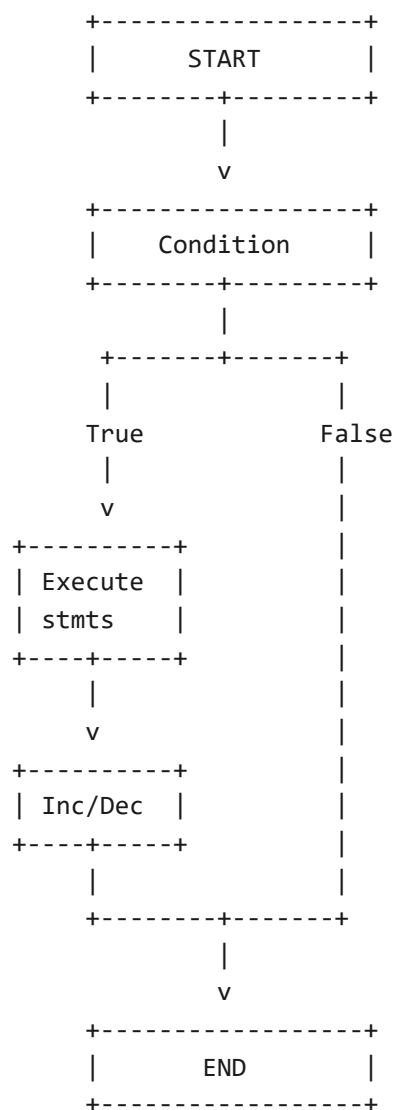
```
Enter a positive integer: 10
Sum of numbers from 1 to 10 is: 55
```

4.4 while Loop

Syntax:

```
while (condition)
{
    // Statements to repeat
    // Increment/Decrement
}
```

Flowchart:



Code:

```
#include <stdio.h>

int main()
{
    int i = 1;

    // Print numbers from 1 to 10
    printf("Numbers from 1 to 10:\n");
    while (i <= 10)
    {
        printf("%d ", i);
        i++;
    }
    printf("\n");

    return 0;
}
```

Output:

```
Numbers from 1 to 10:
1 2 3 4 5 6 7 8 9 10
```

4.5 Program: Reverse a Number using while Loop

```
#include <stdio.h>

int main()
{
    int num, reversed = 0, remainder;

    printf("Enter a number: ");
    scanf("%d", &num);

    while (num != 0)
    {
        remainder = num % 10;
        reversed = reversed * 10 + remainder;
        num = num / 10;
    }

    printf("Reversed number: %d\n", reversed);

    return 0;
}
```

Output:

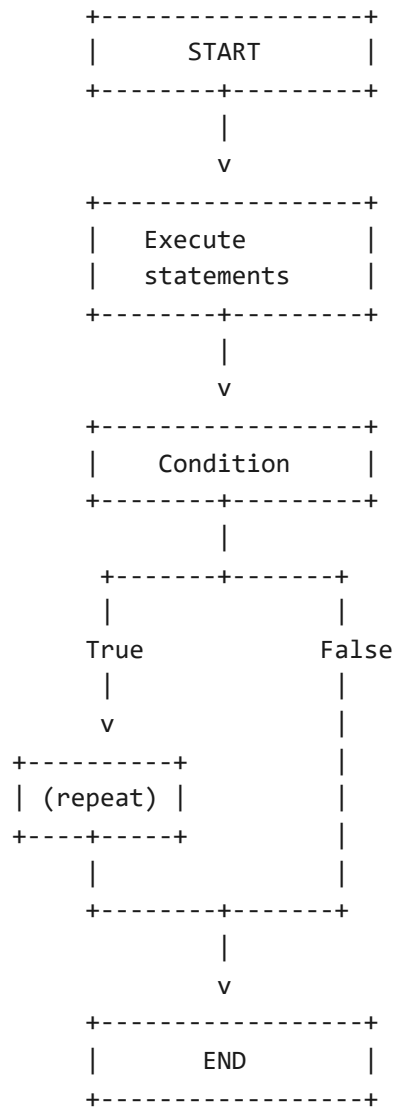
```
Enter a number: 12345
Reversed number: 54321
```

4.6 do-while Loop

Syntax:

```
do
{
    // Statements to repeat
} while (condition);
```

Flowchart:



Code:

```
#include <stdio.h>

int main()
{
    int i = 1;

    // Print numbers from 1 to 10
    printf("Numbers from 1 to 10:\n");
    do
    {
        printf("%d ", i);
        i++;
    } while (i <= 10);
    printf("\n");

    return 0;
}
```

Output:

```
Numbers from 1 to 10:
1 2 3 4 5 6 7 8 9 10
```

4.7 Program: Menu using do-while Loop

```
#include <stdio.h>

int main()
{
    int choice, a, b;

    do
    {
        printf("\n--- MENU ---\n");
        printf("1. Addition\n");
        printf("2. Subtraction\n");
        printf("3. Multiplication\n");
        printf("4. Division\n");
```

```
printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

if (choice >= 1 && choice <= 4)
{
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
}

switch (choice)
{
    case 1:
        printf("Result: %d\n", a + b);
        break;
    case 2:
        printf("Result: %d\n", a - b);
        break;
    case 3:
        printf("Result: %d\n", a * b);
        break;
    case 4:
        if (b != 0)
            printf("Result: %.2f\n", (float)a / b);
        else
            printf("Division by zero!\n");
        break;
    case 5:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice!\n");
}
} while (choice != 5);

return 0;
}
```

Output:

```
--- MENU ---
1. Addition
2. Subtraction
3. Multiplication
```

```
4. Division
5. Exit
Enter your choice: 1
Enter two numbers: 10 5
Result: 15
```

```
--- MENU ---
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 5
Exiting...
```

Topic 5: Nested Control Statements

5.1 Nested Loops

A loop inside another loop is called a **nested loop**.

5.2 Program: Multiplication Table using Nested Loops

```
#include <stdio.h>

int main()
{
    int i, j;

    printf("Multiplication Table (1 to 5):\n");
    for (i = 1; i <= 5; i++)
    {
        for (j = 1; j <= 5; j++)
        {
            printf("%4d", i * j);
        }
        printf("\n");
    }

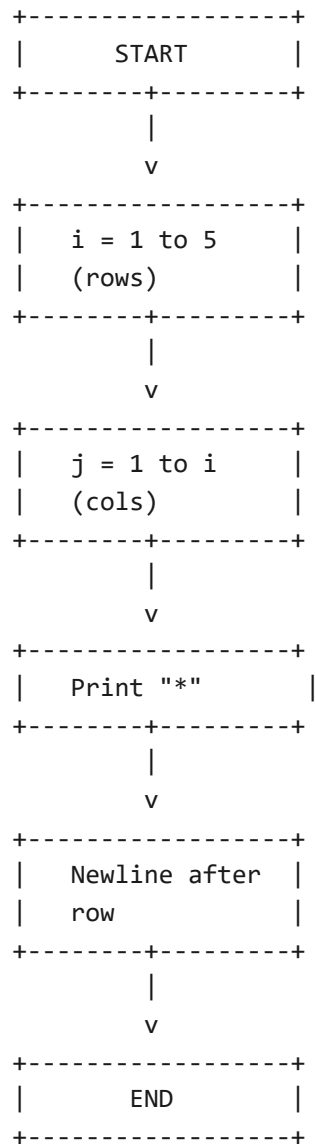
    return 0;
}
```

Output:

```
Multiplication Table (1 to 5):
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
```

5.3 Program: Pattern Printing using Nested Loops

Flowchart:



Code:

```
#include <stdio.h>

int main()
{
    int i, j;

    // Pattern 1: Right triangle
    printf("Pattern 1:\n");
    for (i = 1; i <= 5; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("* ");
        }
        printf("\n");
    }

    printf("\n");

    // Pattern 2: Inverted right triangle
    printf("Pattern 2:\n");
    for (i = 5; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
        {
            printf("* ");
        }
        printf("\n");
    }

    printf("\n");

    // Pattern 3: Number triangle
    printf("Pattern 3:\n");
    for (i = 1; i <= 5; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}
```

```
    }  
  
    return 0;  
}
```

Output:

Pattern 1:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Pattern 2:

```
* * * * *  
* * * *  
* * *  
* *  
*
```

Pattern 3:

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

Topic 6: Break, Continue, Goto Statements

6.1 break Statement

Purpose: Exits from a loop or switch statement immediately.

```
#include <stdio.h>

int main()
{
    int i;

    printf("Loop with break at 5:\n");
    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
            break;
        printf("%d ", i);
    }
    printf("\nLoop terminated.\n");

    return 0;
}
```

Output:

```
Loop with break at 5:
1 2 3 4
Loop terminated.
```

6.2 continue Statement

Purpose: Skips the current iteration and continues with the next iteration.

```
#include <stdio.h>

int main()
```



```
{  
    int i;  
  
    printf("Loop with continue at 5:\n");  
    for (i = 1; i <= 10; i++)  
    {  
        if (i == 5)  
            continue;  
        printf("%d ", i);  
    }  
    printf("\nLoop completed.\n");  
  
    return 0;  
}
```

Output:

```
Loop with continue at 5:  
1 2 3 4 6 7 8 9 10  
Loop completed.
```

6.3 goto Statement

Purpose: Jumps to a labeled statement.

```
#include <stdio.h>

int main()
{
    int num;

    printf("Enter a positive number: ");
    scanf("%d", &num);

    if (num < 0)
        goto negative;
    else
        goto positive;

negative:
    printf("Number is negative.\n");
    goto end;

positive:
    printf("Number is positive.\n");

end:
    printf("End of program.\n");

    return 0;
}
```

Output:

```
Enter a positive number: -5
Number is negative.
End of program.
```

6.4 Comparison Table

Statement	Purpose	Use Case
break	Exit loop/switch	Terminate loop early
continue	Skip iteration	Skip specific conditions
goto	Jump to label	Error handling (rare)

Topic 7: Comma Operator

7.1 What is Comma Operator?

The comma operator `,` is used to **combine multiple expressions** into a single expression. It evaluates left to right.

7.2 Example of Comma Operator

```
#include <stdio.h>

int main()
{
    int a, b, c;

    // Comma operator in for loop
    printf("Using comma in for loop:\n");
    for (a = 1, b = 10; a <= b; a++, b--)
    {
        printf("a = %d, b = %d\n", a, b);
    }

    // Comma operator in variable initialization
    c = (a = 5, b = 10, a + b);
    printf("\nComma operator result: %d\n", c);

    return 0;
}
```

Output:

```
Using comma in for loop:
a = 1, b = 10
a = 2, b = 9
a = 3, b = 8
a = 4, b = 7
a = 5, b = 6
```

Comma operator result: 15

Topic 8: Arrays

8.1 What is an Array?

An array is a **collection of similar data elements** stored in contiguous memory locations under a single name.

8.2 Array Declaration

```
data_type array_name[size];
```

Example:

```
int marks[5];           // Array of 5 integers
float prices[10];       // Array of 10 floats
char name[20];          // Array of 20 characters
```

8.3 Array Initialization

```
// Method 1: Compile-time initialization
int marks[5] = {10, 20, 30, 40, 50};

// Method 2: Partial initialization
int marks[5] = {10, 20, 30}; // Remaining elements = 0

// Method 3: Without size
int marks[] = {10, 20, 30, 40, 50}; // Size = 5

// Method 4: Runtime initialization
int marks[5];
for (i = 0; i < 5; i++)
{
    scanf("%d", &marks[i]);
}
```

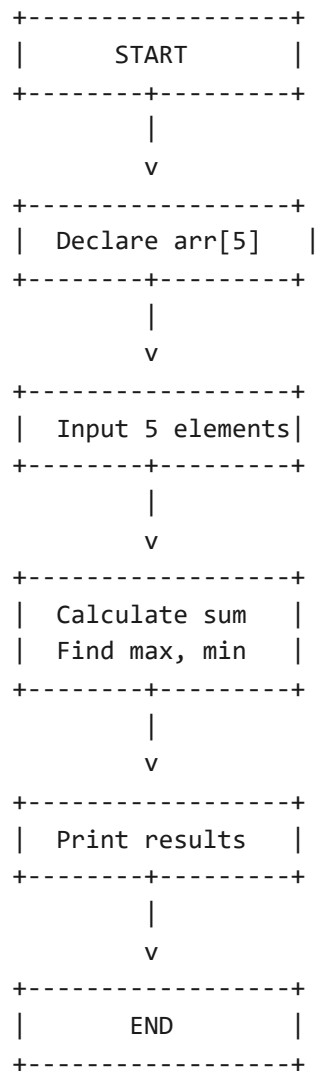
8.4 Accessing Array Elements

Array elements are accessed using **index** (0 to size-1).

```
marks[0] = 10;    // First element  
marks[1] = 20;    // Second element  
marks[4] = 50;    // Last element
```

8.5 Program: Array Operations

Flowchart:



Code:

```
#include <stdio.h>

int main()
{
    int arr[5];
    int i, sum = 0, max, min;
```



```
// Input array elements
printf("Enter 5 numbers:\n");
for (i = 0; i < 5; i++)
{
    printf("arr[%d] = ", i);
    scanf("%d", &arr[i]);
}

// Initialize max and min
max = min = arr[0];

// Process array
for (i = 0; i < 5; i++)
{
    sum += arr[i];
    if (arr[i] > max)
        max = arr[i];
    if (arr[i] < min)
        min = arr[i];
}

// Print results
printf("\nArray elements: ");
for (i = 0; i < 5; i++)
{
    printf("%d ", arr[i]);
}
printf("\nSum = %d", sum);
printf("\nAverage = %.2f", (float)sum / 5);
printf("\nMaximum = %d", max);
printf("\nMinimum = %d\n", min);

return 0;
}
```

Output:

```
Enter 5 numbers:
arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40
arr[4] = 50
```

Array elements: 10 20 30 40 50

Sum = 150

Average = 30.00

Maximum = 50

Minimum = 10

Topic 9: Passing Arrays to Functions

9.1 Passing Array to Function

```
// Function prototype
void display(int arr[], int size);

// Function definition
void display(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }
}

// Function call
display(numbers, 5);
```

9.2 Program: Passing Array to Function

```
#include <stdio.h>

// Function to find sum of array elements
int sumArray(int arr[], int size)
{
    int sum = 0;
    for (int i = 0; i < size; i++)
    {
        sum += arr[i];
    }
    return sum;
}

// Function to print array elements
void printArray(int arr[], int size)
{
    printf("Array elements: ");
```

```
    for (int i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main()
{
    int numbers[5] = {10, 20, 30, 40, 50};
    int total;

    printArray(numbers, 5);
    total = sumArray(numbers, 5);
    printf("Sum = %d\n", total);

    return 0;
}
```

Output:

```
Array elements: 10 20 30 40 50
Sum = 150
```

Topic 10: Multidimensional Arrays

10.1 Two-Dimensional Array

Declaration:

```
data_type array_name[rows][columns];
```

Example:

```
int matrix[3][3]; // 3x3 matrix
```

10.2 2D Array Initialization

```
// Method 1: Complete initialization
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

// Method 2: Without inner braces
int matrix[2][3] = {1, 2, 3, 4, 5, 6};

// Method 3: Partial initialization
int matrix[2][3] = {{1, 2}, {4}};
```

10.3 Program: Matrix Operations

```
#include <stdio.h>

int main()
{
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int i, j, sum = 0;
```

```
printf("Matrix:\n");
for (i = 0; i < 3; i++)
{
    for (j = 0; j < 3; j++)
    {
        printf("%4d", matrix[i][j]);
        sum += matrix[i][j];
    }
    printf("\n");
}

printf("Sum of all elements = %d\n", sum);

// Sum of diagonal elements
printf("Diagonal elements: ");
for (i = 0; i < 3; i++)
{
    printf("%d ", matrix[i][i]);
}
printf("\n");

return 0;
}
```

Output:

```
Matrix:
1 2 3
4 5 6
7 8 9
Sum of all elements = 45
Diagonal elements: 1 5 9
```

Topic 11: Arrays and Strings

11.1 String as Character Array

A string is a **character array** ending with null character `\0`.

Declaration:

```
char str[20] = "Hello";  
char str[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

11.2 Program: String Operations

```
#include <stdio.h>  
#include <string.h>  
  
int main()  
{  
    char str1[50] = "Hello";  
    char str2[50] = "World";  
    char str3[100];  
    int length;  
  
    // String length  
    length = strlen(str1);  
    printf("Length of '%s' = %d\n", str1, length);  
  
    // String copy  
    strcpy(str3, str1);  
    printf("str3 = %s\n", str3);  
  
    // String concatenation  
    strcat(str1, " ");  
    strcat(str1, str2);  
    printf("Concatenated string: %s\n", str1);  
  
    // String comparison  
    int result = strcmp(str2, "World");
```

```
    printf("strcmp(str2, \"World\") = %d\\n", result);  
  
    return 0;  
}
```

Output:

```
Length of 'Hello' = 5  
str3 = Hello  
Concatenated string: Hello World  
strcmp(str2, "World") = 0
```


Topic 12: HPU Important Questions

12.1 Short Answer Questions (2-5 Marks)

1. What are control statements? Name their types.
2. Explain if-else statement with example.
3. What is switch statement? Give syntax.
4. What are loops? Explain for loop.
5. What is the difference between while and do-while loop?
6. What is the purpose of break and continue statements?
7. What is an array? How is it declared?
8. What is the difference between break and continue?
9. What is nested loop? Give example.
10. What is the comma operator?

12.2 Long Answer Questions (10 Marks)

1. Explain all branching statements in C with examples.
2. Compare and contrast while, do-while, and for loops.
3. Explain switch statement with example.
4. What are arrays? Explain one-dimensional and two-dimensional arrays.
5. What is the difference between break, continue, and goto?

12.3 Program Writing Questions (5 Marks)

1. Write a program to find the largest of three numbers using if-else.
2. Write a program to print multiplication table using for loop.
3. Write a program to find the sum of array elements.
4. Write a program to check if a number is prime using loops.
5. Write a program to print patterns using nested loops.

Quick Revision

Loop Comparison

Feature	for	while	do-while
Entry/Exit	Entry	Entry	Exit
Initialization	Inside	Outside	Outside
Counter	Automatic	Manual	Manual
Min Executions	0	0	1
Use Case	Fixed iterations	Variable iterations	Menu-driven

Array Summary

Type	Declaration	Example
1D Array	<code>int arr[5];</code>	<code>int marks[5] = {10,20,30};</code>
2D Array	<code>int arr[3][3];</code>	<code>int matrix[2][2] = {{1,2},{3,4}};</code>
String	<code>char str[20];</code>	<code>char name[] = "Hello";</code>

End of Unit-III Notes